

34. テンプレ識別パイプライン入口

solar_ws0129-main

2026-07-29

対象

項目	内容
ファイル	scripts/run_identification_pipeline.py
種別	Python
区分	identification
役割	template package の raw データから地図生成・基礎整備・識別処理を繋ぐ入口。
起動文脈	identify action で呼ばれる。

このファイルを読む理由

template package の raw データから地図生成・基礎整備・識別処理を繋ぐ入口。

- template/package 初期整備向きの高位入口。

呼び出し関係

どこから呼ばれるか

- scripts/solar_control.sh

次に読むと理解が進むファイル

- scripts/run_vehicle_identification.py

同一リポジトリ内の主要依存

- mpc_solarcar/solar_profile.py

ソース冒頭の把握

モジュール docstring または先頭意図の要約:

モジュール docstring は明示されていない。

代表コード断片

```
from mpc_solarcar.solar_profile import get_section, load_profile

def run(cmd):
    print('RUN', ' '.join(cmd))
    subprocess.run(cmd, check=True)

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument('--profile_yaml', required=True)
    ap.add_argument('--input_dir', default='')
    ap.add_argument('--output_dir', default='')
    args = ap.parse_args()

    profile_path, cfg = load_profile(args.profile_yaml)
    ident_cfg = get_section(cfg, 'identification')
    input_dir = os.path.abspath(args.input_dir or ident_cfg.get('input_dir', 'data/identification/raw'))
    output_dir = os.path.abspath(args.output_dir or ident_cfg.get('output_dir', 'outputs/identification'))
    os.makedirs(output_dir, exist_ok=True)

    python = sys.executable
```

主要構造の読み取り

主要関数は run, main。CLI 引数宣言は 3 件。

主要クラス・関数

行	種別	名前	補足
15	function	run	args: cmd
20	function	main	

ファイルを上から読んだときの定義順

行	内容
8	ROOT に Path(__file__).resolve().parents[1] の結果を代入する。
9	条件 str(ROOT) not in sys.path を判定し、真なら内部処理を行う。
15	関数 run を定義する。
20	関数 main を定義する。
88	条件 __name__ == '__main__' を判定し、真なら内部処理を行う。

import 群の詳細

行	import 文	このファイルで読む理由
---	----------	-------------

2	<code>import argparse</code>	CLI 引数を宣言し、実行時パラメータを外から受け取るため。このファイル内での主な使用位置は L21。
3	<code>import os</code>	パス、環境変数、プロセス外部状態を扱うため。このファイル内での主な使用位置は L29, L30, L31, L35, L36, L40, L41, L44, ...。
4	<code>import subprocess</code>	git 情報取得や外部コマンド実行を行うため。このファイル内での主な使用位置は L17。
5	<code>import sys</code>	sys モジュールを利用するため。このファイル内での主な使用位置は L9, L10, L33。
6	<code>from pathlib import Path</code>	ファイルやディレクトリを安全に扱うため。このファイル内での主な使用位置は L8。
12	<code>from mpc_solarcar.solar_profile import get_section, load_profile</code>	profile YAML 読込と検証から <code>get_section</code> , <code>load_profile</code> を読み込み、このファイルの内部処理を分担させるため。実体ファイルは <code>mpc_solarcar/solar_profile.py</code> 。このファイル内での主な使用位置は L27, L28, L72。

関数・クラスを上から順に解説

L15 関数 `run`

- 定義: `run(cmd)`
 - docstring: 明示されていない。
 - このブロックが直接呼ぶ主な関数・メソッド: `join`, `print`, `run`
 - 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
1. `print(...)` を実行する。
 2. `subprocess.run(...)` を実行する。

L20 関数 `main`

- 定義: `main()`
 - docstring: 明示されていない。
 - このブロックが直接呼ぶ主な関数・メソッド: `ArgumentParser`, `abspath`, `add_argument`, `exists`, `get`, `get_section`, `join`, `load_profile`, `makedirs`, `parse_args`, `print`, `run`
 - 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
1. `ap` に `argparse.ArgumentParser()` の結果を代入する。
 2. `ap.add_argument(...)` を実行する。
 3. `ap.add_argument(...)` を実行する。
 4. `ap.add_argument(...)` を実行する。
 5. `args` に `ap.parse_args()` の結果を代入する。
 6. `(profile_path, cfg)` に `load_profile(args.profile_yaml)` の結果を代入する。
 7. `ident_cfg` に `get_section(cfg, 'identification')` の結果を代入する。

8. `input_dir` に `os.path.abspath(args.input_dir or ident_cfg.get('input_dir', 'data/identification/raw'))` の結果を代入する。
9. `output_dir` に `os.path.abspath(args.output_dir or ident_cfg.get('output_dir', 'outputs/identification'))` の結果を代入する。
10. `os.makedirs(...)` を実行する。

CLI 引数

行	引数
22	<code>--profile_yaml</code>
23	<code>--input_dir</code>
24	<code>--output_dir</code>

処理の流れ

1. CLI 引数を解釈し、`main()` から処理を起動する。

このファイルの位置づけ

この資料群では、`scripts/run_identification_pipeline.py` を **identification** の中核として扱う。したがって、ここで扱う処理は単独で完結せず、前段の `profile / launch / telemetry` と後段の `planner / logger / report` へ繋がる。